Performance Optimization of Fast Fourier Transform (FFT) on Multi-Node Systems

Lu Lu
Data Center GPU Software Solutions
eelu.lu@amd.com

ABSTRACT

Fast Fourier Transform (FFT) is a well-known algorithm that calculates the discrete Fourier Transform of discrete data, and an essential tool in scientific and engineering computation. Due to the large amounts of data, parallelly executing FFT in Graphics Processing Unit (GPU) can effectively optimize the performance. In this paper, we propose some optimized methods to accelerate FFT on the heterogeneous system. For a single GPU, we propose the memory optimized method based on share memory and texture memory to reduce the number of global memory access to achieve better efficiency. The access mode is also optimized to coordinate with the GPU memory access pattern. In distributed memory computation, we use a slab decomposition for FFT. To reduce the all-to-all communication time, an asynchronous hipStream API and unblocking MPI are adopted to respectively optimize the intra-node and inter-node communication. Our preliminary results show that our method can outperform rocFFT, and the optimization of communication has a great potential.

Keywords: Faster Fourier Transform (FFT), GPU, Performance Optimization, Communication.

1 Introduction

The FFT has been applied extensively to a wide range of applications, ranging from image filtering to electronic structure calculations, thus improving the performance of FFT is of great significance. Nowadays, the heterogeneous platform is gradually applied in a variety of fields and lots of optimized implementations of FFT have been proposed on CPU platform, the GPU platform and other accelerator platforms.

Since there is still considerable room for improvement in the proposed work [1], we introduce our optimization of FFT on multi-node systems in this paper to deal with the memory bottleneck caused by unique memory access pattern of FFT.

In this work, we focus on how to make full use of GPU and reduce the all-to-all communication to accelerate FFT algorithm. For a single GPU, the way of data access has been optimized and a novel way based on Matrix Cores is also demonstrated. For multi-node systems, we implement 3D FFT which consists of C++ and ROCm kernels with HIP peer-to-peer API for intra-node communication and (ROCm-aware) MPI for inter-node communication. Our library now supports 1D and 2D FFT in a wide range of sizes, and the evaluation shows that our method in a single GPU can outperform rocFFT, and the distributed FFT can get a certain speedup.

2 FFT Algorithm

2.1 Baseline FFT Algorithm

The normal Discrete Fourier Transform (DFT) incur a high computation cost with an asymptotic complexity of $O(N^2)$. In cases where N is significantly large, direct computation affects performance of the system adversely. A faster approach, known as FFT, introduced by Cooley and Tukey reduces this complexity to $O(NlogN)$.

The FFT divides an N point DFT into a smaller set of transforms, where each set is computed using a base-radix N_1 such that ${N_1}^p = N$, where $1 \le p < N$. Thus for a radix-2, the DFT can be re-written as:

$$X(k) = \sum_{n=0}^{\frac{N}{2}-1} x(2n) W_N^{2nk} + W^k \sum_{n=0}^{\frac{N}{2}-1} x(2n+1) W_N^{2nk}, \quad (1)$$

where the first summation follows the even and the second follows the odd sample-points.

2.2 FFT in Matrix Form

The complete FFT algorithm consists of multiple processes of combining N_1 subsequences to get the DFT of the original N-point sequence. This process can be expressed in the form of matrix as equation (2).

$$X_{out} = F_{N_1} \cdot (T_{N_1 N_2} \odot X_{in}) \quad (2)$$

where, $\cdot$ denotes matrix product, $\odot$ denotes element-wise product and N_2 equals N/N_1.

X_{out} represents the matrix form of the output N-point DFT of the original sequence.F_{N_1} denotes the radix-N_1 DFT matrix. $T_{N_1 N_2}$ is an $N_1 \times N_2$ twiddle factor matrix, it has the same size as X_{out} and F_{N_1} and $T_{N_1 N_2}$ shown as follows:

$$F_{N_1} = \begin{bmatrix} W_{N_1}^0 & W_{N_1}^0 & \cdots & W_{N_1}^0 \\ W_{N_1}^0 & W_{N_1}^1 & \cdots & W_{N_1}^{N_1-1} \\ \vdots & \vdots & \vdots & \vdots \\ W_{N_1}^0 & W_{N_1}^{N_1-1} & \ldots & W_{N_1}^{(N_1-1)(N_1-1)} \end{bmatrix}$$

$$T_{N_1 N_2} = \begin{bmatrix} W_N^0 & W_N^0 & \cdots & W_N^0 \\ W_N^0 & W_N^1 & \cdots & W_N^{N_2-1} \\ \vdots & \vdots & \vdots & \vdots \\ W_N^0 & W_N^{N_1-1} & \ldots & W_N^{(N_1-1)(N_2-1)} \end{bmatrix}$$

The above content describes how the FFTs of N_1 short sequences are converted into the FFT of a long sequence through matrix multiplication and element-wise

multiplication. Since the FFT of a sequence of length one is itself, the FFT of the original sequence can be obtained through this process of $\log_{N_1} N$ times.

2.3 FFT in the Distributed System

In this section we will discuss the algorithm of performing a 3D FFT using slab-decomposition first, and the process of data exchange including local transpose and all-to-all communication.

Suppose that the size of input data is $N_0 \times N_1 \times N_2$. There are two methods to decompose a 3D FFT into a distributed memory: 1D decomposition and 2D decomposition. As shown in Figure 1, 1D decomposition, which is called slab decomposition, means that the input data is distributed among the first dimension over P tasks. Each task owns a part of data, the size of which is $N_0/P \times N_1 \times N_2$. In the first stage, each task computes a N_0/P-batch of 2D FFTs of size $N_1 \times N_2$. Then an all-to-all exchange is performed to redistribute the data. After this each task gets a slab of size $N_0 \times \hat{N_1}/P \times \hat{N_2}$, where the hats denote that the Fourier Transform has been computed across the last two dimensions. In the next stage, each task will compute a batch of $\hat{N_1}/P \times \hat{N_2}$ 1D FFTs of length N_0. 2D decomposition, which is also called pencil decomposition, means that the tasks are mapped to a 2D matrix with P_0 rows and P_1 columns, and each task will perform 1D FFT three times and all-to-all communication twice. Though the advantage of pencil decomposition is the strong scalability when the cluster is large enough for computation, we choose slab decomposition for 3D FFT because our node scale is small and we can reduce the impact on communication bottlenecks.

The whole process includes two transposes, local transpose and global transpose. Local transpose is to pack the noncontiguous data into a contiguous buffer before executing an all-to-all communication, and doing an unpacking operation to get the data back into its correct format after all-to-all communication. In the global transpose, each task should send a fraction of the data to other tasks and receive the same size of data from these tasks.

Global transpose is usually completed by an all-to-all communication with the Message Passing Interface (MPI), which is also the most expensive part of distributed FFT in the communication phase.

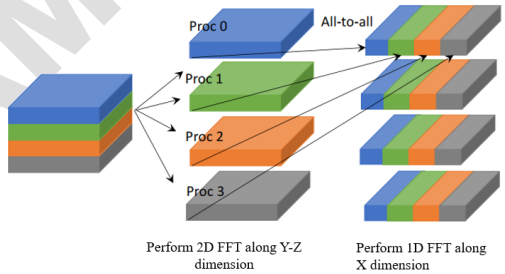

Figure 1. Slab decomposition of the input/output array in two stages of the forward FFT algorithm with one all-to-all communication.

3 Design and Methodology

In this section, we will dive into the overall architecture of our implementation and other optimization approach. The first subsection is the optimization for a single GPU (algorithmic approach), and the second is for multi-GPUs (communication approach).

3.1 Optimization For Single GPU

3.1.1 Optimized Data Access Mode

In our program, the total number of threads is reallocated for the total butterflies in a given stage. Inside the kernel, each butterfly-based thread identifies its position amongst other butterflies using an indexing scheme which we identify as a butterflyclip. Then the results from the calculations are placed back into global memory.

When the data volume is large, the number of threads accesses global memory will increase. But the storage space of share memory is limited. So, when visiting and fetching data, some corresponding changes are made based on the GPU execution to reach the optimum performance. The execution unit of GPU is called warp and each warp has several threads. First, the program divides the data into several pieces, then choose the appropriate number of threads in one block. Second, the program put the data in different pieces into share memory, and the butterfly operation is done in each block. At last, the data in share memory is put back into the global memory. This is a data exchange. If the data was larger, we will choose more times exchange.

During the FFT algorithm, the differences of the butterfly operation in each level are the span and twiddle factor. So, we break the data into pieces to make its span small, and every piece is suitable for computing in the share memory.

3.1.2 Merging Kernels for Matrix-based FFT

As we mentioned in section 2, the FFT algorithm combines radix-N_1 subsequences to get the DFT of the original N-point sequence. We call the process a merging process in the rest of the paper.

Modeling after rocFFT, our implementation uses a simple configuration called a plan. A plan chooses a series of optimal radix-X merging kernels. Then, when the execution function is called, actual transform takes place following the plan [2].

Figure 2 shows the complete process of performing an FFT. First, a function is called to create a plan based on the dimension and the size of the input data. This plan selects an optimal set of merging kernels from the pre-implemented merging kernel collection. Then, the execution function is called with the plan and the original data as input. In the execution function, the previously determined merging

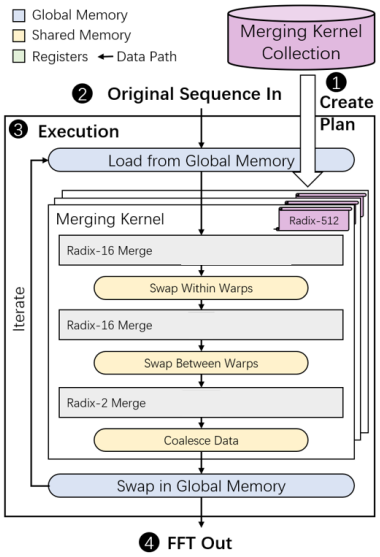

Figure 2. The complete execution process of our implementation.

kernels are executed in turn in multiple iterations. Meantime, multiple types of memory in GPU is fully utilized to increase the reuse of data. After all iterations are executed, the FFT of the original sequence is obtained.

Since Matrix Cores only provide computing power for computing 16 × 16 × 16 GEMM, which can be used in power-of-16 radices, this part of work supports only radix-2 FFT of large input size.

2D FFT performs FFT on each dimension of 2D sequences in turn. It can be implemented by strided batched FFT. Take a 2D FFT on an $N_1 \times N_2$ row-major matrix as an example, an N_1 batch of N_2-point FFT is applied on the N_1 rows and then an N_2 batch of strided N_1-point FFT is applied on the N_1 columns. Therefore, the 2D FFT can be implemented by calling it with proper parameters.

3.2 Optimization for Distributed System

As mentioned above, the computing stages of the parallel 3D FFT algorithm comprise of 2D and 1D FFT on local data. We use rocFFT for this task and each GPU is responsible for one computing task. Each of the two FFTs is done as a single call, combining transforms of multiple sets of data. After finishing the 2D FFT in the first stage, it is necessary to pack data sets that are not contiguous in memory to a buffer memory. Therefore, we implement a kernel function to complete a local data transpose on GPU, which takes up very little time in total communication time. The total process of communication is shown as Figure 3.

Next is globally transposing data between different GPUs. We divide this process into intra-node communication and inter-node communication. For intra-node communication, we use a set of HIP APIs to enable a peer-to-peer memory access within a node, and apply OpenMP to allocate threads to perform parallel computation on different GPUs. An asynchronous memory copy operation with streams is

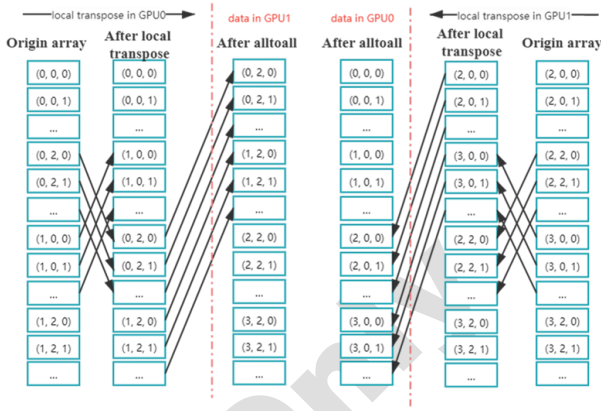

Figure 3. Illustration of the overall communication. In this example, we divide a 4×4×4 3D FFT into two GPUs and the data layout in memory space is z→y→x. We can see that the local transpose makes the noncontiguous data continues before executing an all-to-all communication. After local transpose is an all-to-all communication. Arrows represent an operation of data movement. Between the red dashed lines, we can see the layout of the final global array after redistribution.

applied to perform an intra-node data exchange, which can achieve an overlapping time with inter-node data exchange. For inter-node communication, we use the OpenMPI library with UCX that enables ROCm-aware P2P communication to complete communication between different nodes. Some unblocking point-to-point MPI APIs are adopted to send and receive data asynchronously.

Finally, a local transpose will be performed once again to unpack the data into its correct format. This is followed by a 1D FFT on GPU to complete the whole 3D FFT.

4 Evaluation

In this section, we measure the performance of our FFT implementation on heterogeneous system which equipped multi-node and multi-MI100 GPU.

4.1 Results of FFT in a Single GPU

In this part, we carry out FFT implementation and compare the result with rocFFT.

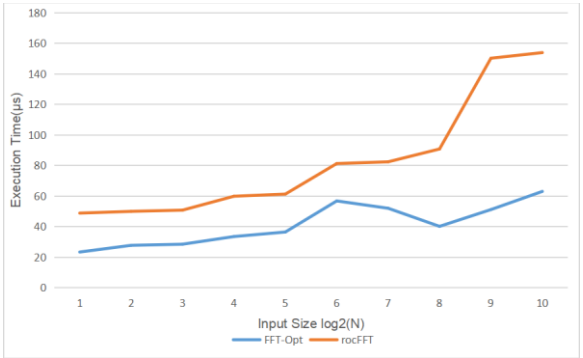

Figure 4. The performance comparison (in microsecond) of our implementation and rocFFT for 1D radix-2 FFT.

Figure 4 shows the performance comparison of 1D radix-2 FFT between our implementation and rocFFT on a MI100 GPU. We call the FFT implementation that applies the above optimized model FFT-Opt. As we can see, the FFT-Opt brings average 1.98x speedup over rocFFT. When the input size gets larger, our implementation performs better, because GPU cannot be fully utilized under small input size.

Figure 6 shows the performance of our matrix-based 2D FFT under large input size, which is introduced in section 3.1.2. Our implementation uses row-major order to store 2D sequences, which means that the data in the first dimension does not continue in memory and the FFT along this dimension is the main performance factor. Our model is obviously faster than rocFFT, when input size gets larger, because the core merging kernels begin to use thread synchronizations which makes some calculations no longer overlap with memory accesses.

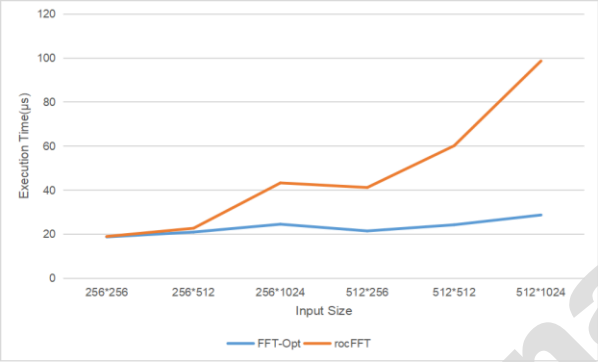

Figure 6. The performance comparison (in microsecond) of our implementation and rocFFT for 2D radix-2 FFT.

4.2 Results of FFT in a distributed system

For distributed FFT, we first examine the performance of our code in one node with different number of GPUs, and then we conduct some experiments on two nodes. Another algorithm for comparison is AccFFT[2], which is also a distributed 3D FFT library and implements all communication using pure MPI.

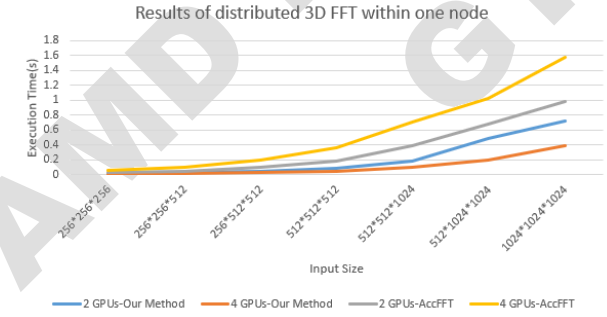

Figure 7. Results of distributed 3D FFT within one node with different number of GPUs.

Figure 7 shows the performance comparison of our distributed 3D FFT and AccFFT within one node. As shown in the figure, our method based on streams to complete communication can achieve better performance, while

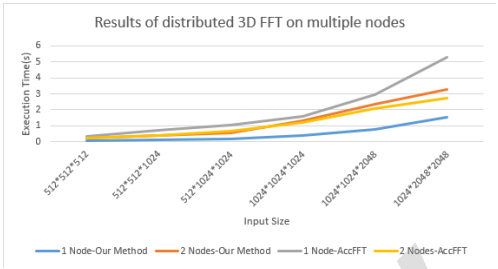

Figure 5. The performance comparison of distributed 3D FFT on multiple nodes.

AccFFT spends too most of the time in PCIe transmission between CPU and GPU. For different number of GPUs within a node, computing on 4 GPUs can also get average 2.03x speedup over 2 GPUs.

Figure 5 shows the performance of distributed 3D FFT on two nodes. We can see that both our method and AccFFT have similar execution time, because when all processes in MPI communicate with each other, the bottleneck of 3D FFT is the network bandwidth instead of the time in PCIe transmission. That is to say, performing 3D FFT of small scale in multiple nodes will be not applicable and it will be a priority when the input size is too large to calculate at once in one node. Therefore, in the future work, we can consider adding a "auto-tune" mode to select proper number of GPUs for different input size and try to add more nodes in the distributed system.

5 Conclusions

In this paper, we introduce some novel ways to improve the FFT parallel performance on the multi-node system under different circumstances of a single GPU and multi-GPU. We evaluate our implementation and rocFFT in various sizes and dimensions on the latest MI100 GPU. Our preliminary results show that our method can outperform rocFFT, and the optimization of inter-node communication has a great potential.

For a single GPU, our current work is inadequate and lack of 3D FFT implementation. We will perfect this part of work and explore other optimizations, like alleviate the memory bottleneck in the future. For distributed 3D FFT on multiple nodes, further optimization will be done to reduce communication time especially the inter-node communication.

References

[1] Durrani, Sultan, et al. "FFT blitz: the tensor cores strike back." Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. 2021.

[2] Gholami, Amir, et al. "AccFFT: A library for distributed-memory FFT on CPU and GPU architectures." arXiv preprint arXiv:1506.07933 (2015).